

Comparing and Totally Ordering Mathematical Expressions: Are traditional processing methods still suitable today?

Hung Dinh Phu Dang

Abstract

I call this the “classic processing workflow”:

$$\begin{array}{ccc} \text{Expressions} & \xrightarrow{\text{Arithmetic Computation}} & \text{Scalar Values} \xrightarrow{\text{Comparison}} \\ \text{Relational Structure and "Spacing between expressions"} (<, >, =) & & \end{array}$$

Observing the center: In many applications, the final scalar value is not what is truly needed; rather, what matters is the relative ordering or distance between expressions. Here, the scalar value merely serves as an intermediate “carrier” for that relational information.

Is the intermediate step of “deriving a scalar value” a mathematically ontological necessity for transitioning from an expression to an ordering or distance relationship? Or does there exist an alternative mathematical space that allows for the extraction of such a relationship directly from the expression’s structure, thereby bypassing the scalar derivation step—either partially or entirely?

This is not a proposal to eliminate computation. The goal is to redefine the concept of computation to encompass structural comparisons that do not necessarily pass through intermediate scalar values.

1 Definition

Axiom A1 — Comparability

There exists a relational operator $\leq_S$, defined directly on the structure of two elements $A, B \in S$, such that:

$$\leq_S (A, B) \iff \text{eval}(A) \leq \text{eval}(B)$$

In other words, $\leq_S$ is logically equivalent to the standard ordering relations $\{<, >, =\}$ on scalar values—but the left-hand side does not involve the explicit, full computation of $\text{eval}(A)$ and $\text{eval}(B)$.

Axiom A2 — Structural Distance / Gap Measure

There exists a function

$$d_S : S \times S \rightarrow \mathbb{R}_{\geq 0}$$

(or an equivalent ordering structure), defined directly on the expression, such that $d_S(A, B)$ accurately reflects the difference $|\text{eval}(A) - \text{eval}(B)|$ (or a monotonic transformation thereof) without requiring the full computation of $\text{eval}(A)$ and $\text{eval}(B)$.

Axiom A3 — Total Order: sufficiently strong, yet without inheriting the costs associated with $\mathbb{R}$.

S must support a total ordering relation with expressive power equivalent to that of $\mathbb{R}$ (sufficient to distinguish every pair of distinct finite-precision values required by the problem).

To clarify the constraint of “avoiding the problems associated with $\mathbb{R}$ ”: this is not a constraint regarding the nature of the set itself (such as cardinality, countability, or the finite representability of $\mathbb{R}$ on computers—an issue effectively resolved in practice by finite-precision floating-point arithmetic). The real constraint lies in the processing cost—COST_ALL (defined in Section 3): if S is simply forced to reduce to $\mathbb{R}$ before a comparison can be made, it reverts exactly to the “classical processing workflow” that the problem aims to avoid, rendering the initial objective cost-ineffective—not “set-theoretically impossible,” but rather “cost-wise trivial or futile.”

Therefore, A3 is understood as: S must possess a total order with a discriminative power equivalent to that of $\mathbb{R}$, while the process of extracting order or distance information (A1, A2) must incur a COST_ALL that is genuinely lower than that of the “reduce to $\mathbb{R}$ and then compare” approach.

Additional condition — contextualizing costs

A1 and A2 must be feasible within a “real-world environment and specified equipment”—that is, subject to specific hardware or computational model constraints, rather than within an idealized computational model with infinite resources. The “total actual implementation cost” (COST_ALL, Section 3) serves as the metric for feasibility—not merely theoretical feasibility (where existence alone suffices), but practical feasibility (where COST_ALL shows a genuine reduction compared to the baseline).

2 Two major issues

Major Issue 1 — The General Case

Given: N expressions, each containing only basic arithmetic operations $\{+, -, \times, \div\}$ with finite-precision real number operands.

Exclusion: All transcendental functions. According to Richardson’s Theorem (1968), determining whether $\text{eval}(A) = 0$ is undecidable in the general case when expressions are permitted to contain transcendental functions; thus, this exclusion is a necessary condition for the problem to remain decidable, rather than an arbitrary choice by the author.

Open question: Does there exist a system S —along with algorithms to com-

pute $\leq_S$ and d_S satisfying axioms A1–A3 for this finite-precision expression class $\{+, -, \times, \div\}$ —that achieves a lower COST_ALL than the baseline?

Major Issue 2 — The Inner Product Case

Given: N expressions of a specific (non-arbitrary) form:

$$\langle u, v \rangle = \sum_{i=1}^n u_i v_i$$

where u_i and v_i are finite-precision real numbers.

Open question: Given the stricter structural constraints (a fixed sum-of-products form rather than an arbitrary expression), are problems A1–A3 easier to solve than Major Problem 1? In other words, does the specific sum-of-products structure allow for a feasible S that might not exist in the general case?

3 Định nghĩa COST_ALL

$$\text{COST_ALL} = \text{Cost}_{\text{computation}} + \text{Cost}_{\text{memory/representation}} + \text{Cost}_{\text{khác}}$$

measured in a real-world environment using specific hardware—not in a computational model that completely abstracts away the resources.

Illustration via the Unary vs. Binary example

(This is an example from the border)

Unary system: represents a value V as a string of V characters; for example,
 $i + j \rightarrow \underbrace{11 \dots 1}_{i+j \text{ characters}}$

In this system, Comparability and Distance (A1, A2) become direct comparisons of length or shape, eliminating the need for mechanical addition—arithmetically, $\text{Cost}_{\text{computation}} \rightarrow O(1)$. However, $\text{Cost}_{\text{memory}} = O(V)$: for $C = 1000 \times 1000$, the Unary system requires a sequence of 10^6 characters—entailing significant memory and unrolling costs.

Scalar/Binary system: $\text{Cost}_{\text{memory}} \approx O(\log V)$ (approximately 20 bits for a value of 10^6), but there is a $\text{Cost}_{\text{computation}}$ associated with the step of reducing the value to a scalar before comparison and distance determination are possible.

Observation: These two systems lie at opposite ends of a computation-cost vs. memory-cost trade-off axis within a one-dimensional (unary) representation space.

4 Statement of the Central Open Problem

Does there exist a space S satisfying A1, A2, and A3 such that the COST_ALL (measured within a clearly defined computational model or physical device) required to extract the ordering relationship and distance between expressions in S is strictly less than the COST_ALL of the classical procedure (reducing to scalars and then

comparing)—for (a) the class of general arithmetic expressions $\{+, -, \times, \div\}$ with finite precision (Major Problem 1), and (b) the class of inner product expressions $\langle u, v \rangle$ (Major Problem 2)?

4.1 Why this is an open problem—not an impossibility

(i) There is no proof of general impossibility. No theorem—including Richardson’s Theorem, which merely excludes transcendental functions from the scope of the problem—demonstrates the non-existence of an $\mathbb{S}$ satisfying A1–A3 with a COST_ALL lower than the baseline.

Richardson’s Theorem limits the scope of the problem (excluding transcendental functions to preserve decidability) but does not limit the existence of $\mathbb{S}$ within the remaining scope.

(ii) There is partial precedent—evidence indicating that the non-trivial problem is not impossible:

Regarding Major Issue 1: techniques for exact or certified geometric predicate evaluation (such as interval arithmetic and adaptive-precision arithmetic—exemplified by the work of Shewchuk and Yap in computational geometry) achieve requirement A1 (determining the sign or order of an expression) at a lower average cost than fully reducing the expression to an exact scalar value, although these techniques do not necessarily satisfy all three axioms (A1–A3) in every case.

Regarding Major Issue 2: techniques such as Locality-Sensitive Hashing, random projection (Johnson–Lindenstrauss lemma), and Maximum Inner Product Search (MIPS) achieve approximations of A1/A2 for the inner product problem, with a COST_ALL lower than the baseline across various real-world data scenarios. However, as these are probabilistic or approximate solutions that do not satisfy A3 (requiring an exact total order without error), the question of whether an exact (non-approximate) solution exists remains open.

(iii) The Unary/Binary example merely illustrates a known two-dimensional trade-off; it neither proves nor disproves the existence of the multi-dimensional intersection point sought by the central problem. Extending the concept from a one-dimensional trade-off (unary versus binary operations on natural numbers) to a multi-dimensional structural space—generalized for arithmetic expressions or inner products—constitutes the unresolved core of Major Problems 1 and 2.

4.2 Conclusion on the status of the problem

Based on points (i)–(iii), the central problem in Section 4 should be classified as an open problem: there is no general solution, nor a general counterexample or impossibility theorem, yet there is sufficient partial precedent (in exact geometric computation and in sketching/LSH/MIPS) to indicate that this direction is not trivially impossible. The difficulty varies between the two sub-cases: Major Problem 2 (inner product) possesses a more constrained structure and already has many efficient approximation solutions, making an exact solution more attainable; in contrast, Major Problem 1 (general arithmetic expressions) involves an expression space that is not constrained by a specific form.